

```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

rray11212_raw_data01_path = kagglehub.dataset_download('rray11212/raw-data01')

print('Data source import complete.')
```

```
import os
import shutil
import random
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.utils import resample
```

```
RAW_DATASET = '/kaggle/input/datasets/rray11212/raw-data01'
BALANCED_DATASET = '/kaggle/working/balanced_blood_group'
TRAIN_DIR = '/kaggle/working/train'
TEST_DIR = '/kaggle/working/test'
IMG_SIZE = (150, 150)
BATCH_SIZE = 32
```

```
2026-03-13 15:39:20.470539: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register cuFFT factory:
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1773416360.717545      55 cuda_dnn.cc:8579] Unable to register cuDNN factory: Attempting to register factory for
E0000 00:00:1773416360.796489      55 cuda_blas.cc:1407] Unable to register cuBLAS factory: Attempting to register factory f
W0000 00:00:1773416361.368890      55 computation_placer.cc:177] computation placer already registered. Please check linkage
W0000 00:00:1773416361.368939      55 computation_placer.cc:177] computation placer already registered. Please check linkage
W0000 00:00:1773416361.368942      55 computation_placer.cc:177] computation placer already registered. Please check linkage
W0000 00:00:1773416361.368944      55 computation_placer.cc:177] computation placer already registered. Please check linkage
```

```
import os
import shutil
from sklearn.utils import resample
```

```
RAW_DATASET = '/kaggle/input/datasets/rray11212/raw-data01/dataset_blood_group'
BALANCED_DATASET = '/kaggle/working/balanced_blood_group'
```

```
os.makedirs(BALANCED_DATASET, exist_ok=True)
```

```
classes = os.listdir(RAW_DATASET)
class_files = {}
for c in classes:
    class_dir = os.path.join(RAW_DATASET, c)
    files = [f for f in os.listdir(class_dir) if os.path.isfile(os.path.join(class_dir, f))]
    if files:
        class_files[c] = files

if not class_files:
    raise ValueError("No non-empty class folders found in the dataset. Please check your data.")
```

```
min_count = min(len(files) for files in class_files.values())
```

```
for cls, images in class_files.items():
    if len(images) < min_count:
        continue
    dst = os.path.join(BALANCED_DATASET, cls)
    os.makedirs(dst, exist_ok=True)
    sampled = resample(images, n_samples=min_count, random_state=42, replace=False)
    for img in sampled:
        shutil.copy(os.path.join(RAW_DATASET, cls, img), os.path.join(dst, img))
```

```
import os
import shutil
import random
```

```
BALANCED_DATASET = '/kaggle/working/balanced_blood_group'
TRAIN_DIR = '/kaggle/working/train'
```

```

TEST_DIR = '/kaggle/working/test'

os.makedirs(TRAIN_DIR, exist_ok=True)
os.makedirs(TEST_DIR, exist_ok=True)

summary = {}

for cls in os.listdir(BALANCED_DATASET):
    class_path = os.path.join(BALANCED_DATASET, cls)
    images = [f for f in os.listdir(class_path) if os.path.isfile(os.path.join(class_path, f))]
    random.shuffle(images)
    split_idx = int(0.8 * len(images))
    train_imgs, test_imgs = images[:split_idx], images[split_idx:]
    train_cls_dir = os.path.join(TRAIN_DIR, cls)
    test_cls_dir = os.path.join(TEST_DIR, cls)
    os.makedirs(train_cls_dir, exist_ok=True)
    os.makedirs(test_cls_dir, exist_ok=True)
    train_count, test_count = 0, 0
    for img in train_imgs:
        try:
            shutil.copy(os.path.join(class_path, img), os.path.join(train_cls_dir, img))
            train_count += 1
        except Exception as e:
            print(f"Error copying {img} to train: {e}")
    for img in test_imgs:
        try:
            shutil.copy(os.path.join(class_path, img), os.path.join(test_cls_dir, img))
            test_count += 1
        except Exception as e:
            print(f"Error copying {img} to test: {e}")
    summary[cls] = (train_count, test_count)
    print(f"Class {cls}: {train_count} train, {test_count} test images copied.")

# Print total images in each split
total_train = sum(x[0] for x in summary.values())
total_test = sum(x[1] for x in summary.values())
print(f"\nTotal images: {total_train} train, {total_test} test")

```

```

Class A+: 452 train, 113 test images copied.
Class AB+: 452 train, 113 test images copied.
Class B-: 452 train, 113 test images copied.
Class O+: 452 train, 113 test images copied.
Class B+: 452 train, 113 test images copied.
Class O-: 452 train, 113 test images copied.
Class A-: 452 train, 113 test images copied.
Class AB-: 452 train, 113 test images copied.

```

```
Total images: 3616 train, 904 test
```

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator

TRAIN_DIR = '/kaggle/working/train'
TEST_DIR = '/kaggle/working/test'
IMG_SIZE = (150, 150)
BATCH_SIZE = 32

train_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    TRAIN_DIR, target_size=IMG_SIZE, batch_size=BATCH_SIZE, class_mode="categorical"
)
test_generator = test_datagen.flow_from_directory(
    TEST_DIR, target_size=IMG_SIZE, batch_size=BATCH_SIZE, class_mode="categorical", shuffle=False
)

```

```

Found 3616 images belonging to 8 classes.
Found 904 images belonging to 8 classes.

```

```

import shutil
import os

for d in ['/kaggle/working/train', '/kaggle/working/test']:
    if os.path.exists(d):
        shutil.rmtree(d)

```

```

import os
import shutil
import random

BALANCED_DATASET = '/kaggle/working/balanced_blood_group'
TRAIN_DIR = '/kaggle/working/train'

```

```

TEST_DIR = '/kaggle/working/test'

os.makedirs(TRAIN_DIR, exist_ok=True)
os.makedirs(TEST_DIR, exist_ok=True)

for cls in os.listdir(BALANCED_DATASET):
    class_path = os.path.join(BALANCED_DATASET, cls)
    images = [f for f in os.listdir(class_path) if os.path.isfile(os.path.join(class_path, f))]
    random.shuffle(images)
    split_idx = int(0.8 * len(images))
    train_imgs, test_imgs = images[:split_idx], images[split_idx:]
    train_cls_dir = os.path.join(TRAIN_DIR, cls)
    test_cls_dir = os.path.join(TEST_DIR, cls)
    os.makedirs(train_cls_dir, exist_ok=True)
    os.makedirs(test_cls_dir, exist_ok=True)
    for img in train_imgs:
        shutil.copy(os.path.join(class_path, img), os.path.join(train_cls_dir, img))
    for img in test_imgs:
        shutil.copy(os.path.join(class_path, img), os.path.join(test_cls_dir, img))
print("Train/test split complete.")

```

Train/test split complete.

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator

TRAIN_DIR = '/kaggle/working/train'
TEST_DIR = '/kaggle/working/test'
IMG_SIZE = (150, 150)
BATCH_SIZE = 32

train_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    TRAIN_DIR, target_size=IMG_SIZE, batch_size=BATCH_SIZE, class_mode="categorical"
)
test_generator = test_datagen.flow_from_directory(
    TEST_DIR, target_size=IMG_SIZE, batch_size=BATCH_SIZE, class_mode="categorical", shuffle=False
)
num_classes = train_generator.num_classes
print(f"Classes: {train_generator.class_indices}")

```

Found 3616 images belonging to 8 classes.
 Found 904 images belonging to 8 classes.
 Classes: {'A+': 0, 'A-': 1, 'AB+': 2, 'AB-': 3, 'B+': 4, 'B-': 5, 'O+': 6, 'O-': 7}

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

def create_custom_cnn(num_classes):
    model = Sequential([
        Conv2D(32, (3,3), activation='relu', input_shape=(150,150,3)),
        MaxPooling2D(2,2),
        Conv2D(64, (3,3), activation='relu'),
        MaxPooling2D(2,2),
        Conv2D(128, (3,3), activation='relu'),
        MaxPooling2D(2,2),
        Flatten(),
        Dense(512, activation='relu'),
        Dense(num_classes, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model

model = create_custom_cnn(num_classes)
model.fit(train_generator, epochs=10, validation_data=test_generator, verbose=1)
model.save('/kaggle/working/custom_cnn_model.h5')
acc = model.evaluate(test_generator, verbose=0)[1]
print(f"Custom CNN accuracy: {acc:.4f}")

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2026-03-13 08:41:09.432069: E external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51] failed call to cuInit: INTERNAL
Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyData
self._warn_if_super_not_called()
113/113 ————— 108s 938ms/step - accuracy: 0.3769 - loss: 2.1148 - val_accuracy: 0.7312 - val_loss: 0.6643
Epoch 2/10
113/113 ————— 105s 928ms/step - accuracy: 0.8102 - loss: 0.4780 - val_accuracy: 0.8650 - val_loss: 0.3431
Epoch 3/10
113/113 ————— 141s 923ms/step - accuracy: 0.8766 - loss: 0.3102 - val_accuracy: 0.8794 - val_loss: 0.3024
Epoch 4/10
113/113 ————— 108s 954ms/step - accuracy: 0.8875 - loss: 0.2787 - val_accuracy: 0.9137 - val_loss: 0.2251

```

```
Epoch 5/10
113/113 ————— 108s 957ms/step - accuracy: 0.8929 - loss: 0.2573 - val_accuracy: 0.8872 - val_loss: 0.2877
Epoch 6/10
113/113 ————— 112s 993ms/step - accuracy: 0.9393 - loss: 0.1655 - val_accuracy: 0.9082 - val_loss: 0.2379
Epoch 7/10
113/113 ————— 115s 1s/step - accuracy: 0.9274 - loss: 0.1822 - val_accuracy: 0.8562 - val_loss: 0.3514
Epoch 8/10
113/113 ————— 115s 1s/step - accuracy: 0.9469 - loss: 0.1441 - val_accuracy: 0.9038 - val_loss: 0.2415
Epoch 9/10
113/113 ————— 111s 986ms/step - accuracy: 0.9740 - loss: 0.0818 - val_accuracy: 0.8772 - val_loss: 0.3338
Epoch 10/10
113/113 ————— 142s 983ms/step - accuracy: 0.9552 - loss: 0.1226 - val_accuracy: 0.9060 - val_loss: 0.2216
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file for
Custom CNN accuracy: 0.9060
```

```
import os
import shutil
import random

BALANCED_DATASET = '/kaggle/working/balanced_blood_group'
TRAIN_DIR = '/kaggle/working/train'
TEST_DIR = '/kaggle/working/test'

# Clean up previous splits if needed
for d in [TRAIN_DIR, TEST_DIR]:
    if os.path.exists(d):
        shutil.rmtree(d)

os.makedirs(TRAIN_DIR, exist_ok=True)
os.makedirs(TEST_DIR, exist_ok=True)

for cls in os.listdir(BALANCED_DATASET):
    class_path = os.path.join(BALANCED_DATASET, cls)
    images = [f for f in os.listdir(class_path) if os.path.isfile(os.path.join(class_path, f))]
    random.shuffle(images)
    split_idx = int(0.8 * len(images))
    train_imgs, test_imgs = images[:split_idx], images[split_idx:]
    train_cls_dir = os.path.join(TRAIN_DIR, cls)
    test_cls_dir = os.path.join(TEST_DIR, cls)
    os.makedirs(train_cls_dir, exist_ok=True)
    os.makedirs(test_cls_dir, exist_ok=True)
    for img in train_imgs:
        shutil.copy(os.path.join(class_path, img), os.path.join(train_cls_dir, img))
    for img in test_imgs:
        shutil.copy(os.path.join(class_path, img), os.path.join(test_cls_dir, img))
print("Train/test split complete.")
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
/tmp/ipykernel_55/1208307546.py in <cell line: 0>()
    15 os.makedirs(TEST_DIR, exist_ok=True)
    16
--> 17 for cls in os.listdir(BALANCED_DATASET):
    18     class_path = os.path.join(BALANCED_DATASET, cls)
    19     images = [f for f in os.listdir(class_path) if os.path.isfile(os.path.join(class_path, f))]

FileNotFoundError: [Errno 2] No such file or directory: '/kaggle/working/balanced_blood_group'
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

IMG_SIZE = (150, 150)
BATCH_SIZE = 32
TRAIN_DIR = '/kaggle/working/train'
TEST_DIR = '/kaggle/working/test'

train_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    TRAIN_DIR, target_size=IMG_SIZE, batch_size=BATCH_SIZE, class_mode="categorical"
)
test_generator = test_datagen.flow_from_directory(
    TEST_DIR, target_size=IMG_SIZE, batch_size=BATCH_SIZE, class_mode="categorical", shuffle=False
)
num_classes = train_generator.num_classes
```

```
Found 3616 images belonging to 8 classes.
Found 904 images belonging to 8 classes.
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```

TRAIN_DIR = '/kaggle/working/train'
TEST_DIR = '/kaggle/working/test'
IMG_SIZE = (150, 150)
BATCH_SIZE = 32

train_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    TRAIN_DIR, target_size=IMG_SIZE, batch_size=BATCH_SIZE, class_mode="categorical"
)
test_generator = test_datagen.flow_from_directory(
    TEST_DIR, target_size=IMG_SIZE, batch_size=BATCH_SIZE, class_mode="categorical", shuffle=False
)
num_classes = train_generator.num_classes
print(f"Classes: {train_generator.class_indices}")

```

```

Found 3616 images belonging to 8 classes.
Found 904 images belonging to 8 classes.
Classes: {'A+': 0, 'A-': 1, 'AB+': 2, 'AB-': 3, 'B+': 4, 'B-': 5, 'O+': 6, 'O-': 7}

```

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

def create_shallow_cnn(num_classes):
    model = Sequential([
        Conv2D(16, (3,3), activation='relu', input_shape=(150,150,3)),
        MaxPooling2D(2,2),
        Flatten(),
        Dense(64, activation='relu'),
        Dense(num_classes, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model

model = create_shallow_cnn(num_classes)
model.fit(train_generator, epochs=10, validation_data=test_generator, verbose=1)
model.save('/kaggle/working/shallow_cnn_model.h5')
acc = model.evaluate(test_generator, verbose=0)[1]
print(f"Shallow CNN accuracy: {acc:.4f}")

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDat
self._warn_if_super_not_called()
Epoch 1/10
113/113 ━━━━━━━━━━━ 24s 202ms/step - accuracy: 0.2132 - loss: 4.4190 - val_accuracy: 0.3728 - val_loss: 1.4150
Epoch 2/10
113/113 ━━━━━━━━━━━ 23s 205ms/step - accuracy: 0.5564 - loss: 1.1221 - val_accuracy: 0.6327 - val_loss: 0.9788
Epoch 3/10
113/113 ━━━━━━━━━━━ 23s 202ms/step - accuracy: 0.7909 - loss: 0.6109 - val_accuracy: 0.7046 - val_loss: 0.7850
Epoch 4/10
113/113 ━━━━━━━━━━━ 24s 209ms/step - accuracy: 0.9217 - loss: 0.3215 - val_accuracy: 0.7113 - val_loss: 0.7776
Epoch 5/10
113/113 ━━━━━━━━━━━ 23s 205ms/step - accuracy: 0.9650 - loss: 0.1825 - val_accuracy: 0.7046 - val_loss: 0.8435
Epoch 6/10
113/113 ━━━━━━━━━━━ 23s 206ms/step - accuracy: 0.9796 - loss: 0.1228 - val_accuracy: 0.7246 - val_loss: 0.7377
Epoch 7/10
113/113 ━━━━━━━━━━━ 40s 202ms/step - accuracy: 0.9970 - loss: 0.0564 - val_accuracy: 0.7290 - val_loss: 0.8064
Epoch 8/10
113/113 ━━━━━━━━━━━ 24s 209ms/step - accuracy: 0.9983 - loss: 0.0308 - val_accuracy: 0.7257 - val_loss: 0.8298
Epoch 9/10
113/113 ━━━━━━━━━━━ 23s 205ms/step - accuracy: 1.0000 - loss: 0.0197 - val_accuracy: 0.7389 - val_loss: 0.8080
Epoch 10/10
113/113 ━━━━━━━━━━━ 23s 205ms/step - accuracy: 1.0000 - loss: 0.0127 - val_accuracy: 0.7423 - val_loss: 0.8314
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file for
Shallow CNN accuracy: 0.7423

```

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense, Dropout

def create_mlp(num_classes):
    model = Sequential([
        Flatten(input_shape=(150,150,3)),
        Dense(256, activation='relu'),
        Dropout(0.5),
        Dense(128, activation='relu'),
        Dropout(0.5),
        Dense(num_classes, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model

```

```

model = create_mlp(num_classes)
model.fit(train_generator, epochs=10, validation_data=test_generator, verbose=1)
model.save('/kaggle/working/mlp_model.h5')
acc = model.evaluate(test_generator, verbose=0)[1]
print(f"MLP accuracy: {acc:.4f}")

```

```

Epoch 1/10
113/113 ————— 23s 193ms/step - accuracy: 0.1321 - loss: 14.7341 - val_accuracy: 0.1250 - val_loss: 2.0795
Epoch 2/10
113/113 ————— 22s 192ms/step - accuracy: 0.1234 - loss: 2.1498 - val_accuracy: 0.1250 - val_loss: 2.0795
Epoch 3/10
113/113 ————— 21s 189ms/step - accuracy: 0.1200 - loss: 2.0814 - val_accuracy: 0.1250 - val_loss: 2.0795
Epoch 4/10
113/113 ————— 21s 189ms/step - accuracy: 0.1171 - loss: 2.0905 - val_accuracy: 0.1250 - val_loss: 2.0795
Epoch 5/10
113/113 ————— 22s 195ms/step - accuracy: 0.1291 - loss: 2.0827 - val_accuracy: 0.1250 - val_loss: 2.0795
Epoch 6/10
113/113 ————— 21s 190ms/step - accuracy: 0.1280 - loss: 2.0796 - val_accuracy: 0.1250 - val_loss: 2.0795
Epoch 7/10
113/113 ————— 21s 188ms/step - accuracy: 0.1267 - loss: 2.0792 - val_accuracy: 0.1250 - val_loss: 2.0795
Epoch 8/10
113/113 ————— 21s 190ms/step - accuracy: 0.1133 - loss: 2.0797 - val_accuracy: 0.1250 - val_loss: 2.0795
Epoch 9/10
113/113 ————— 41s 188ms/step - accuracy: 0.1212 - loss: 2.0796 - val_accuracy: 0.1250 - val_loss: 2.0794
Epoch 10/10
113/113 ————— 21s 190ms/step - accuracy: 0.1279 - loss: 2.0796 - val_accuracy: 0.1250 - val_loss: 2.0794
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file for
MLP accuracy: 0.1250

```

```

from tensorflow.keras.applications import ResNet50, VGG16, EfficientNetB0, MobileNetV2
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.models import Model

```

```

def create_transfer_learning_model(base_model_name, num_classes):
    if base_model_name == 'ResNet50':
        base_model = ResNet50(weights=None, include_top=False, input_shape=(150,150,3))
    elif base_model_name == 'VGG16':
        base_model = VGG16(weights=None, include_top=False, input_shape=(150,150,3))
    elif base_model_name == 'EfficientNetB0':
        base_model = EfficientNetB0(weights=None, include_top=False, input_shape=(150,150,3))
    elif base_model_name == 'MobileNetV2':
        base_model = MobileNetV2(weights=None, include_top=False, input_shape=(150,150,3))
    else:
        raise ValueError(f"Unknown model: {base_model_name}")
    base_model.trainable = False
    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(512, activation='relu')(x)
    x = Dropout(0.5)(x)
    predictions = Dense(num_classes, activation='softmax')(x)
    model = Model(inputs=base_model.input, outputs=predictions)
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model

```

```

for model_name in ['ResNet50', 'VGG16', 'EfficientNetB0', 'MobileNetV2']:
    print(f"\nTraining {model_name}...")
    model = create_transfer_learning_model(model_name, num_classes)
    model.fit(train_generator, epochs=10, validation_data=test_generator, verbose=1)
    model.save(f'/kaggle/working/{model_name}_model.h5')
    acc = model.evaluate(test_generator, verbose=0)[1]
    print(f"{model_name} accuracy: {acc:.4f}")

```

```

Training ResNet50...
Epoch 1/5
113/113 ————— 169s 1s/step - accuracy: 0.1257 - loss: 2.0949 - val_accuracy: 0.1250 - val_loss: 2.0766
Epoch 2/5
113/113 ————— 159s 1s/step - accuracy: 0.1335 - loss: 2.0779 - val_accuracy: 0.1527 - val_loss: 2.0765
Epoch 3/5
113/113 ————— 160s 1s/step - accuracy: 0.1420 - loss: 2.0761 - val_accuracy: 0.1250 - val_loss: 2.0755
Epoch 4/5
113/113 ————— 165s 1s/step - accuracy: 0.1369 - loss: 2.0759 - val_accuracy: 0.1847 - val_loss: 2.0693
Epoch 5/5
113/113 ————— 198s 1s/step - accuracy: 0.1629 - loss: 2.0675 - val_accuracy: 0.2478 - val_loss: 2.0629
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file for
ResNet50 accuracy: 0.2478

```

```

Training VGG16...
Epoch 1/5
113/113 ————— 465s 4s/step - accuracy: 0.1166 - loss: 2.0800 - val_accuracy: 0.1250 - val_loss: 2.0791
Epoch 2/5
113/113 ————— 466s 4s/step - accuracy: 0.1238 - loss: 2.0789 - val_accuracy: 0.1748 - val_loss: 2.0788
Epoch 3/5
113/113 ————— 512s 5s/step - accuracy: 0.1231 - loss: 2.0788 - val_accuracy: 0.1261 - val_loss: 2.0784
Epoch 4/5

```

```

113/113 ————— 465s 4s/step - accuracy: 0.1341 - loss: 2.0789 - val_accuracy: 0.1726 - val_loss: 2.0778
Epoch 5/5
113/113 ————— 463s 4s/step - accuracy: 0.1430 - loss: 2.0783 - val_accuracy: 0.1294 - val_loss: 2.0770
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file for
VGG16 accuracy: 0.1294

Training EfficientNetB0...
Epoch 1/5
113/113 ————— 86s 662ms/step - accuracy: 0.1243 - loss: 2.0796 - val_accuracy: 0.1250 - val_loss: 2.0795
Epoch 2/5
113/113 ————— 75s 663ms/step - accuracy: 0.1173 - loss: 2.0797 - val_accuracy: 0.1250 - val_loss: 2.0795
Epoch 3/5
113/113 ————— 66s 581ms/step - accuracy: 0.1258 - loss: 2.0796 - val_accuracy: 0.1250 - val_loss: 2.0795
Epoch 4/5
113/113 ————— 65s 579ms/step - accuracy: 0.1092 - loss: 2.0798 - val_accuracy: 0.1250 - val_loss: 2.0795
Epoch 5/5
113/113 ————— 68s 599ms/step - accuracy: 0.1130 - loss: 2.0796 - val_accuracy: 0.1250 - val_loss: 2.0795
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file for
EfficientNetB0 accuracy: 0.1250

Training MobileNetV2...
Epoch 1/5
113/113 ————— 49s 387ms/step - accuracy: 0.1159 - loss: 2.0798 - val_accuracy: 0.1250 - val_loss: 2.0795
Epoch 2/5
113/113 ————— 44s 389ms/step - accuracy: 0.1118 - loss: 2.0797 - val_accuracy: 0.1250 - val_loss: 2.0794
Epoch 3/5
113/113 ————— 40s 356ms/step - accuracy: 0.1183 - loss: 2.0797 - val_accuracy: 0.1250 - val_loss: 2.0795
Epoch 4/5
113/113 ————— 40s 346ms/step - accuracy: 0.1218 - loss: 2.0797 - val_accuracy: 0.1250 - val_loss: 2.0795
Epoch 5/5
113/113 ————— 39s 349ms/step - accuracy: 0.1180 - loss: 2.0797 - val_accuracy: 0.1250 - val_loss: 2.0794
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file for
MobileNetV2 accuracy: 0.1250

```

```

import shutil
shutil.make_archive('/kaggle/working/working_dir_backup', 'zip', '/kaggle/working')

from IPython.display import FileLink, display
display(FileLink('/kaggle/working/working_dir_backup.zip'))

```

[/kaggle/working/working_dir_backup.zip](#)